

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Introduction
Coordinate Frames
IMU Fusion

Oct 01, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

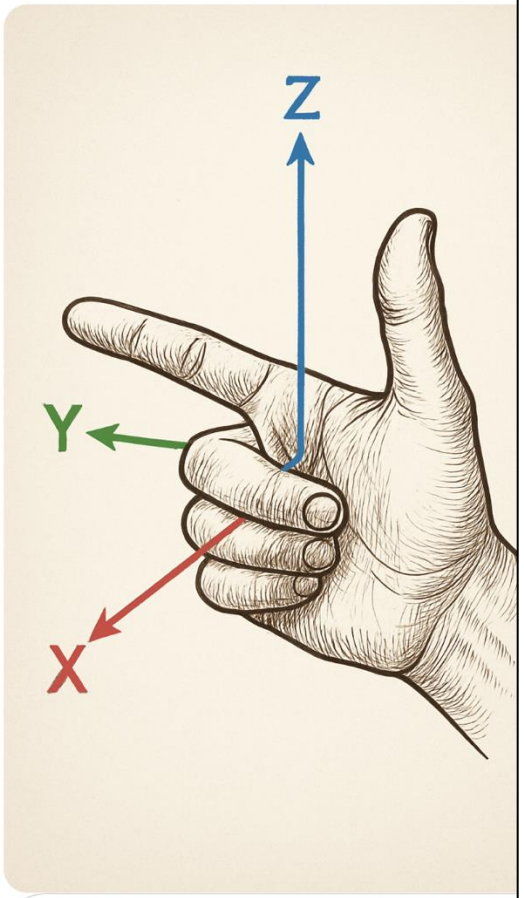
Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **Materials provided on Moodle starting on Monday, Oct 06:**
  - Slides for the lecture
  - Formulary
  - Python code
  - Exercises / Solutions provided after discussion in the lecture
  - Exercises are optional, one sheet per week, but are highly recommended.
    They can be handed in for correction.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

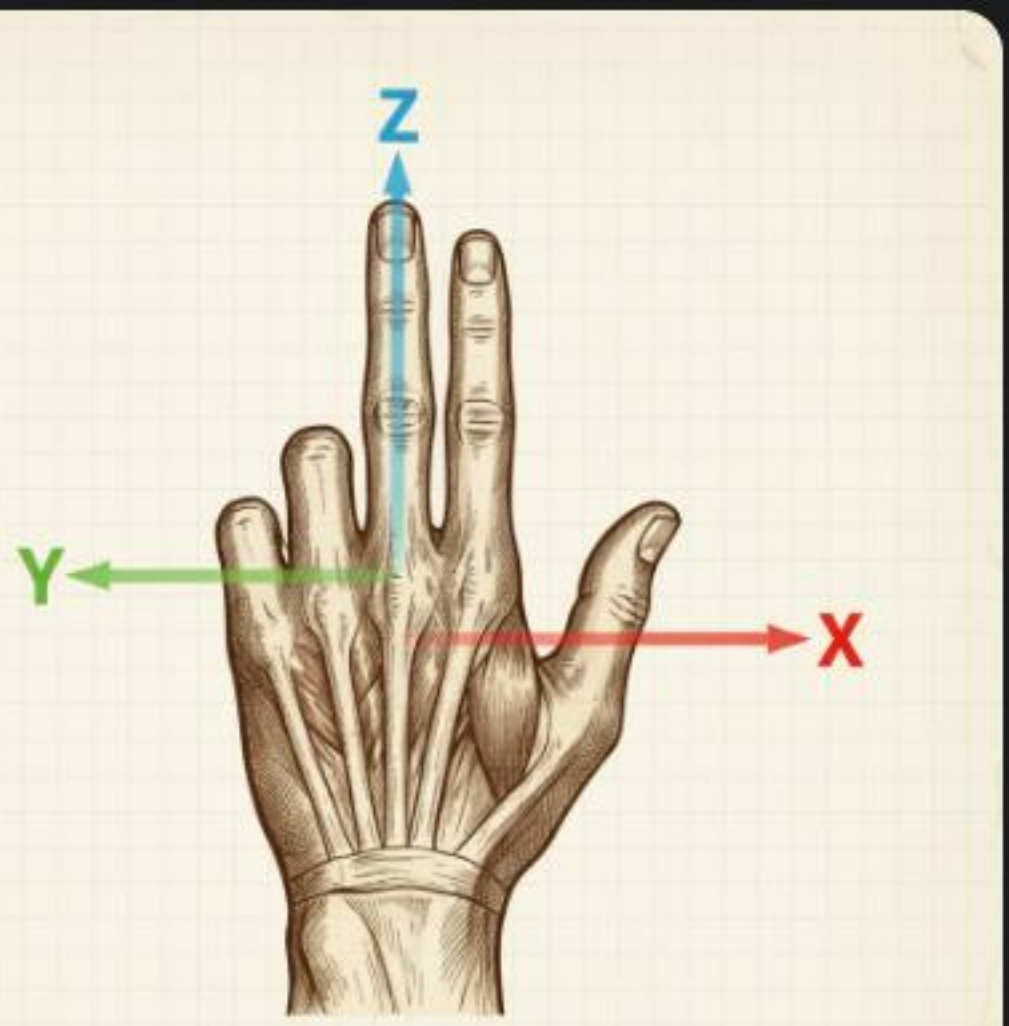
Sensor Networks and Sensor Fusion

About the use of AI.

mach mir ein
koordinatensy
weg in den Bil
nach oben. Ze
blau. Der Stil



Hier ist deine Visualisierung des rechtshändigen Koordinatensystems:



Rechtshändiges Koorlinatensystem

- **We encourage the use of AI for accelerating your work.**
- **However, AI still makes mistakes.**
- **If you do not know well what you are doing, these mistakes can be very hard to detect and correct.**
- **Do not rely on AI unless you know what you‘re doing, and are ready to double-check.**

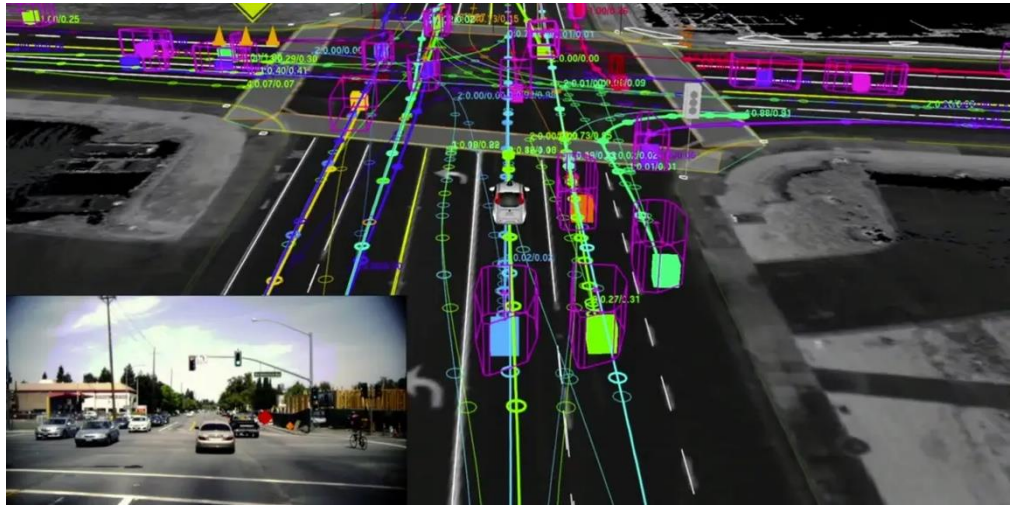
What we will cover in this lecture series:

- **Getting input from several sensors: IMUs, cameras, lidars**
- **Interrelating sensor data in space and time: Coordinate systems, tracking, prediction, innovation**
- **Working with objects and point clouds obtained from different sensors**
- **Localizing and navigating with external landmarks and in self-generated maps**
- **Unfortunately a lot of quite involved mathematics...**

- **Motivation: Why do we fuse data of different sensors?**
- **Interrelating data from different sensors: Geometric and Temporal**
  - Coordinate frames and coordinate transformations
  - The role of ego motion
- **A simple example: Roll / pitch estimation from Accelerometer and Gyro**

Motivation
Why do we fuse sensor data?

- **End Goal:**
  We Want To Automate A Real-World Task With A Robot.
- **For any automation task operating in the real world, the robot needs a world model that captures the state of the real world accurately enough.**
- **If the task is sufficiently complex, chances are that there is no single sensor able to gather enough data for the world model.**
- **Example for object detection:**
  - Cameras are 2D sensors and cannot measure distance or speed.
  - Radars cannot determine the class of an object.
  - Lidars distort the shape of moving objects and cannot measure speed.

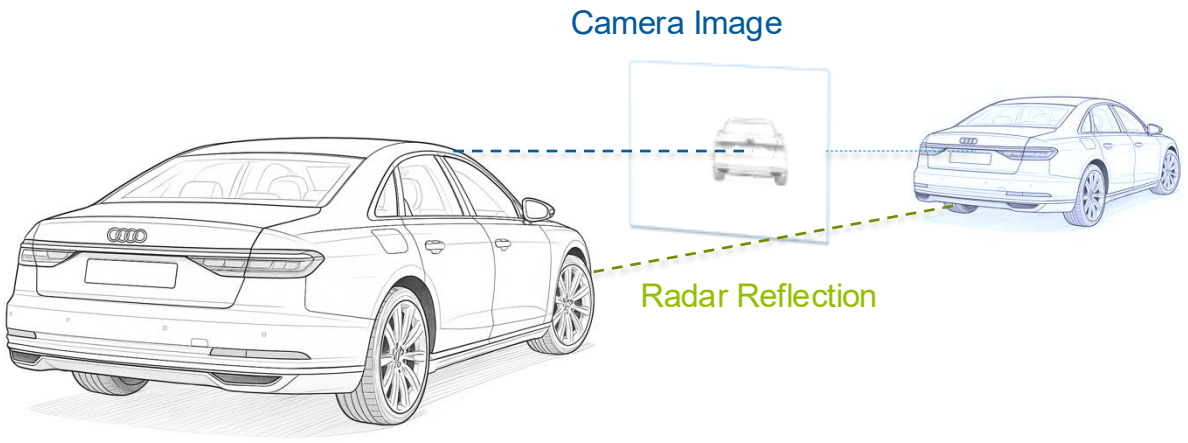






World models are used by self-driving cars and humanoid robots alike.

- **Fusing the output of multiple complementary sensors can generate a sufficient world model.**
- **E.g.:**
  - Detecting a car in a camera video stream (azimuth and elevation)
    $$m_{\text{cam}} = (\alpha_{\text{cam}}, \beta_{\text{cam}})$$
  - Detecting an object in a radar target list (azimuth and distance)
    $$m_{\text{rad}} = (\alpha_{\text{rad}}, d)$$
  - Correlating both if they are the same object
    $$|\alpha_{\text{cam}} - \alpha_{\text{rad}}| < \alpha_{\text{correl}}$$
  - Determining 3D coordinates
    $$\text{x} = (d \sin \alpha_{cam}, d \sin \beta_{cam}, d)T$$



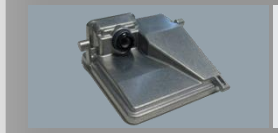
Sensor Networks and Sensor Fusion

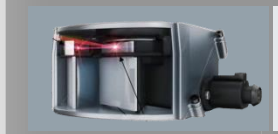
Motivation

- **Modern robots almost all use several types of exteroceptive and introceptive sensors.**
- **Many different fusion algorithms and principles – we will look at most of them.**

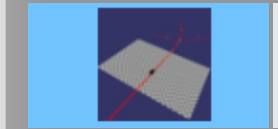


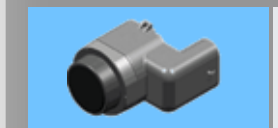

Long Range Radar


Video


Lidar

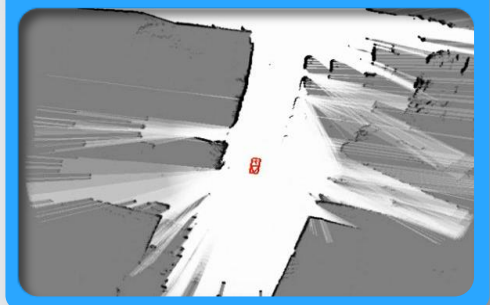

Mid-Range Radar


Map data

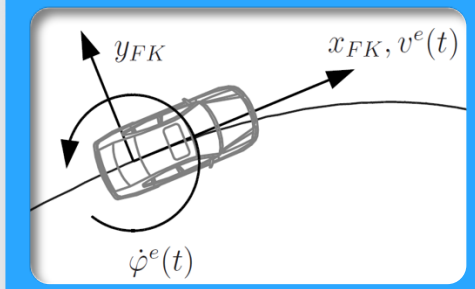

Sonar


Surround-view camera

Grid fusion



Object fusion



Obstacle detection

Leading vehicle detection

Path planning

Actor control

Function control

Sensor Networks and Sensor Fusion

Motivation

(Auto-)mobile applications

- Localization/navigation
- Tracking of traffic participants
- Interior monitoring for crash-mitigations
- …..

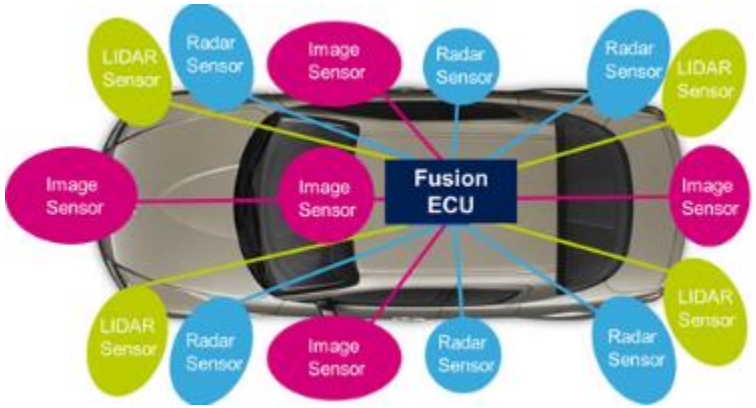
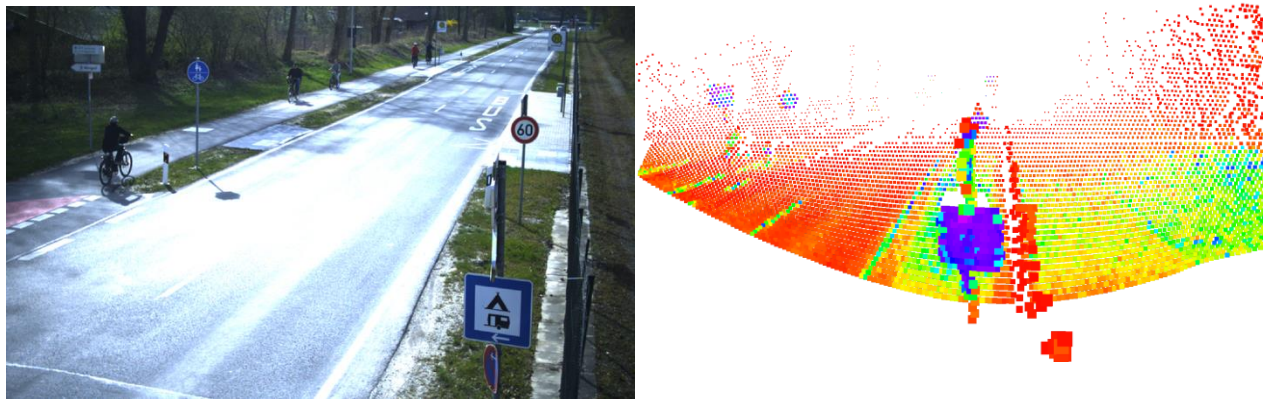
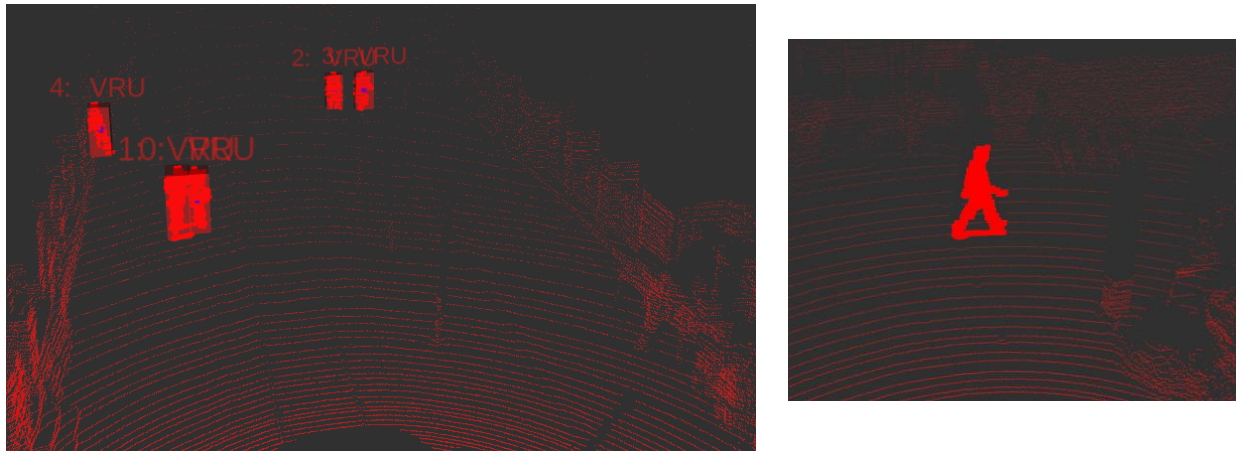


Figure borrowed from [1]



Camera-image (left) and Lidar-point cloud (right) of a Road Side Unit,Source: Fraunhofer IVI



Tracking of pedestrians fusing camera images and Lidar data (Quelle: Fraunhofer IVI)

[1] https://www.fierceelectronics.com/components/three-sensor-types-drive-autonomous-vehicles

Security and Defence

- Navigation and Tracking
- Situation Awareness
- ….

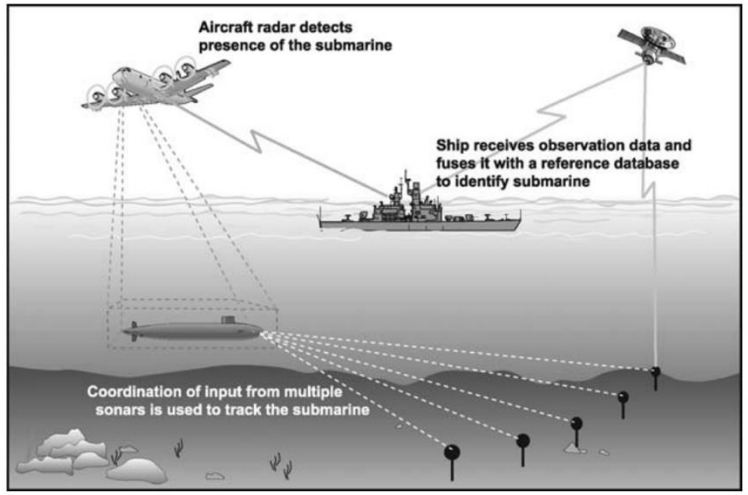


Figure borrowed from [2]

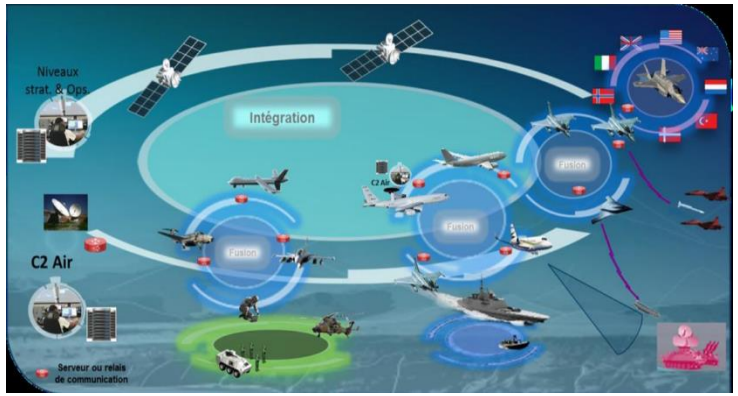


Figure borrowed from [3]

[1] https://www.fierceelectronics.com/components/three-sensor-types-drive-autonomous-vehicles
[2] D.L. Hall, J. Llinas: Introduction to Multisensor Data Fusion
[3] P. Gros: The tactical cloud – a key element of the future combat air system, Recherche Stratégique, 2019

Consumer-Applications

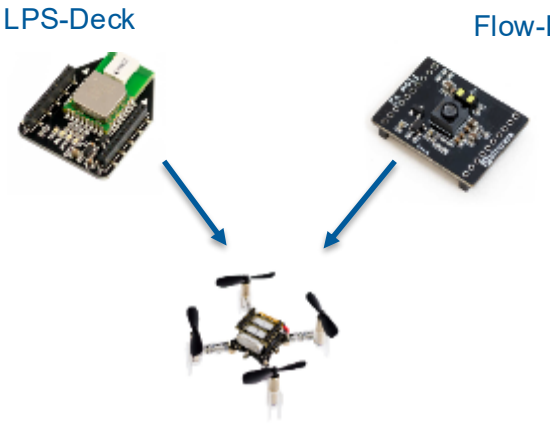
- Smartphone-applications (e.g. indoor navigation)
- Smartwatches/Fitnesstracker
- Drone applications
- …



Innenraum-Navigation



Smart-Watch/Fitness-Tracker



Crazyflie 2.1 Mini-Quadcopter

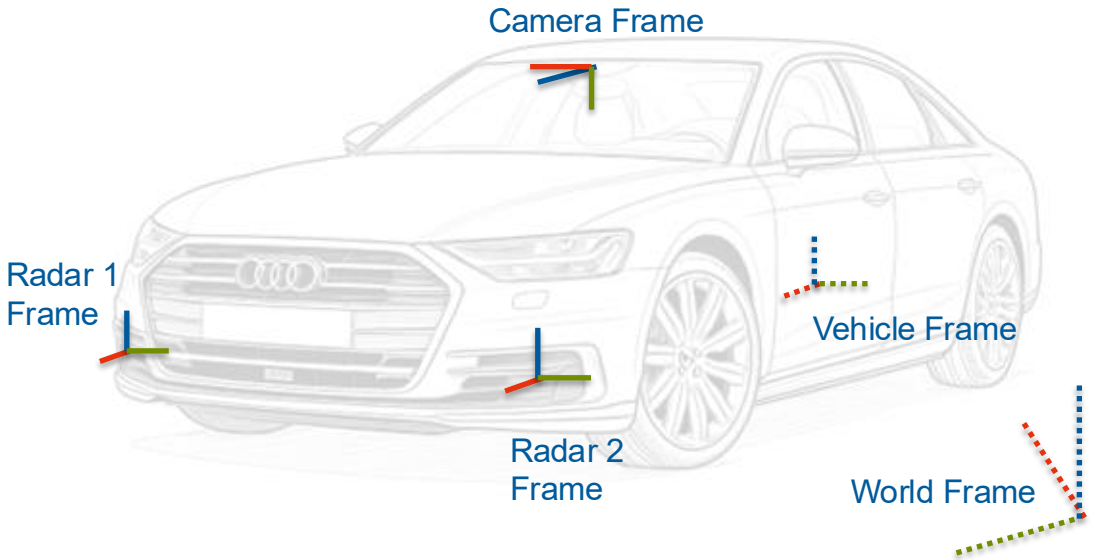
[1] https://www.fierceelectronics.com/components/three-sensor-types-drive-autonomous-vehicles
[2] D.L. Hall, J. Llinas: Introduction to Multisensor Data Fusion
[3] P. Gros: The tactical cloud – a key element of the future combat air system, Recherche Stratégique, 2019

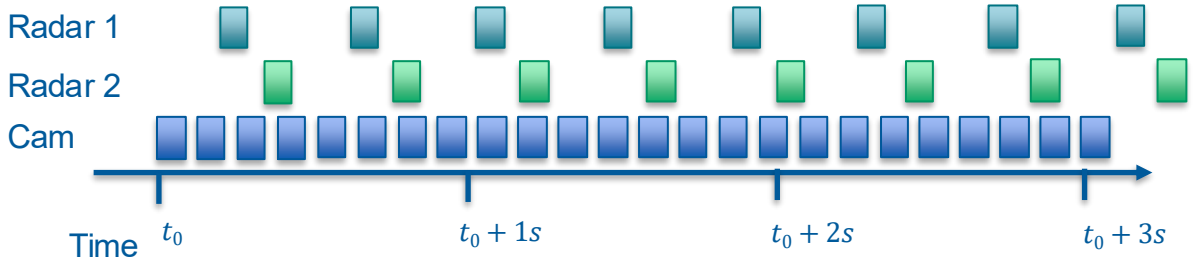
Interrelating Data from Different Sensors

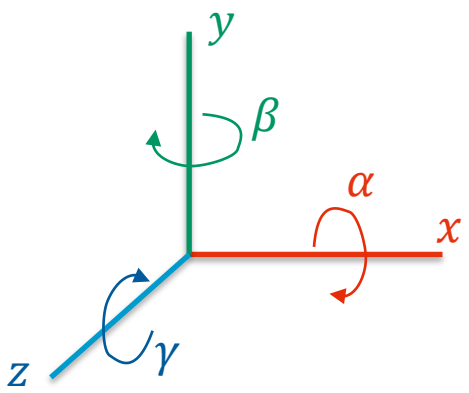
Sensor Networks and Sensor Fusion

Interrelating Data from Different Sensors

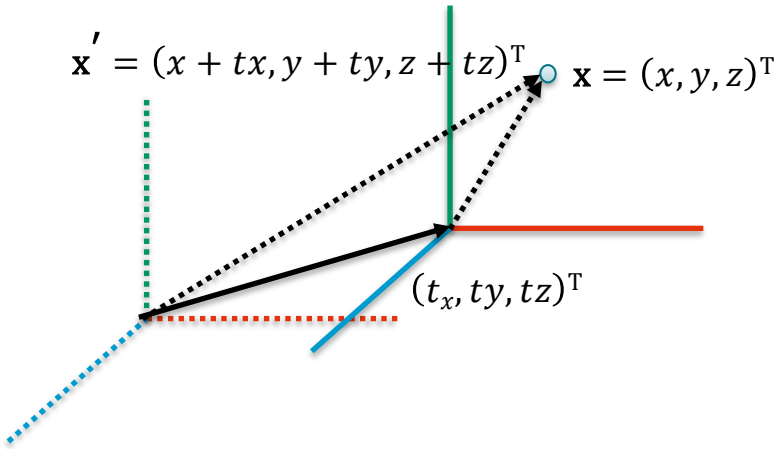
- **In a multisensor system, sensors are typically geometrically and temporally diverse.**
- **Every sensor provides its output in**
  - its own **coordinate frame**, determined by how and where it is mounted,
  - its own **time frame**, determined by its measuring method, bandwidth etc.
- **Many aspects only make sense to be fused in the World Coordinate Frame.**
- **E.g. absolute velocity of measured objects – if a leading vehicle does not move in the radar frame, how fast is that vehicle**
  - if the ego vehicle is standing?
  - if the ego vehicle is going 200kph?
- **We will concentrate on coordinate frames for now, and leave temporal coordination as an exercise.**







- We almost always work in right-hand coordinate systems – x is the thumb, y the index, z the middle finger.
- We designate α, β, γ as rotation around the x, y, z axes respectively.



- The same 3D point can be represented in different coordinate systems or **frames.**
- We relate frames with **coordinate transformations.**
- For pure translation (= normal vectors of coordinate systems all parallel), a coordinate transformation is just a vector addition:

  $\mathbf{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ and $x' = \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}$ describe the same point

  in two coordinate frames that are offset by $\begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}$.

- Vector $\mathbf{v} \in R^n$:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

- Matrix $\mathbf{A} \in R^{m \times n}$:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

- Multiplying a vector $\mathbf{v} \in R^n$ with a matrix $\mathbf{A} \in R^{m \times n}$:

$$\mathbf{w} = \mathbf{A}\mathbf{v} = \begin{pmatrix} a_{11}v_1 + a_{12}v_2 + \cdots + a_{1n}v_n \\ a_{21}v_1 + a_{22}v_2 + \cdots + a_{2n}v_n \\ \vdots \\ a_{m1}v_1 + a_{m2}v_2 + \cdots + a_{mn}v_n \end{pmatrix}$$

Only defined if dimensions match, i.e. the number of columns in the matrix is equal to the number of rows in the vector!

- Multiplying two matrices $\mathbf{A} \in R^{m \times n}$ and $\mathbf{B} \in R^{n \times m}$:

$$\mathbf{C} = \mathbf{AB} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} + \cdots + a_{1n}b_{n1} & a_{11}b_{12} + a_{12}b_{22} + \cdots + a_{1n}b_{n2} & \dots & a_{11}b_{1m} + a_{12}b_{2m} + \cdots + a_{1n}b_{nm} \\ a_{21}b_{11} + a_{22}b_{21} + \cdots + a_{2n}b_{n1} & a_{21}b_{12} + a_{22}b_{22} + \cdots + a_{2n}b_{n2} & \dots & a_{21}b_{1m} + a_{22}b_{2m} + \cdots + a_{2n}b_{nm} \\ \dots & \dots & \ddots & \dots \\ a_{m1}b_{11} + a_{m2}b_{21} + \cdots + a_{mn}b_{n1} & a_{m1}b_{12} + a_{m2}b_{22} + \cdots + a_{mn}b_{n2} & \dots & a_{m1}b_{1m} + a_{m2}b_{2m} + \cdots + a_{mn}b_{nm} \end{pmatrix}$$

  Only defined if dimensions match, i.e.

  - the number of columns in matrix **A** is equal to the number of rows in matrix **B**
  - and the number of rows in matrix **A** is equal to the number of columns in matrix **B**!
- Matrix multiplication is **associative**: **A(BC)=(AB)C**, but **not commutative: AB != BA**
- Special matrix: Unity matrix (or submatrix)

$$\mathbf{I^{n \times n}} = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

2D Rotation

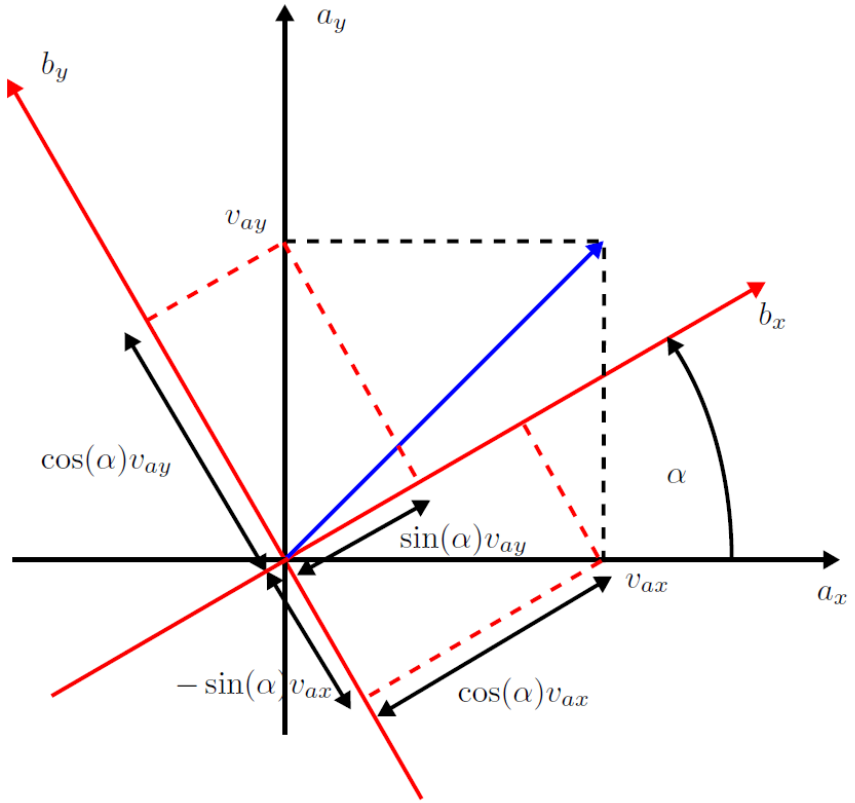
- We want to express a vector $v_a = [v_{ax} \quad v_{ay}]^T$ given in frame a in the coordinates of the frame b: $v_b = [v_{bx} \quad v_{by}]^T$
- Frame b is rotated with respect to frame a by an angle α
- Compute the projection of v_{ax} and v_{ay} on the axis of frame b

$$v_{b,x} = \cos(\alpha)v_{a,x} + \sin(\alpha)v_{a,y}$$
$$v_{b,y} = -\sin(\alpha)v_{a,x} + \cos(\alpha)v_{a,y}$$

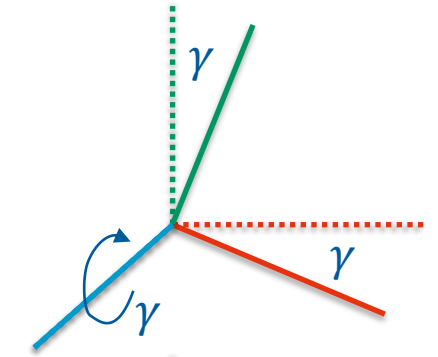
- Transferring the above to matrix calculus we obtain

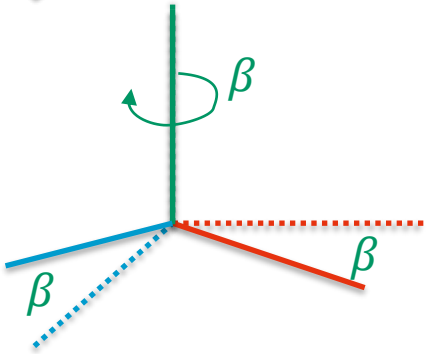
$$\boldsymbol{v}_b = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{bmatrix} \cdot \begin{bmatrix} v_{a,x} \\ v_{a,y} \end{bmatrix}$$

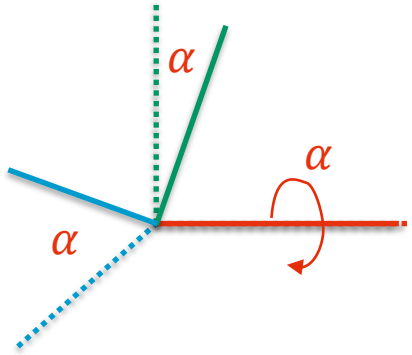
Rotational matrix $\boldsymbol{T}_a^b$ in 2D



Representing Rotations







- Elementary rotation by γ around z:

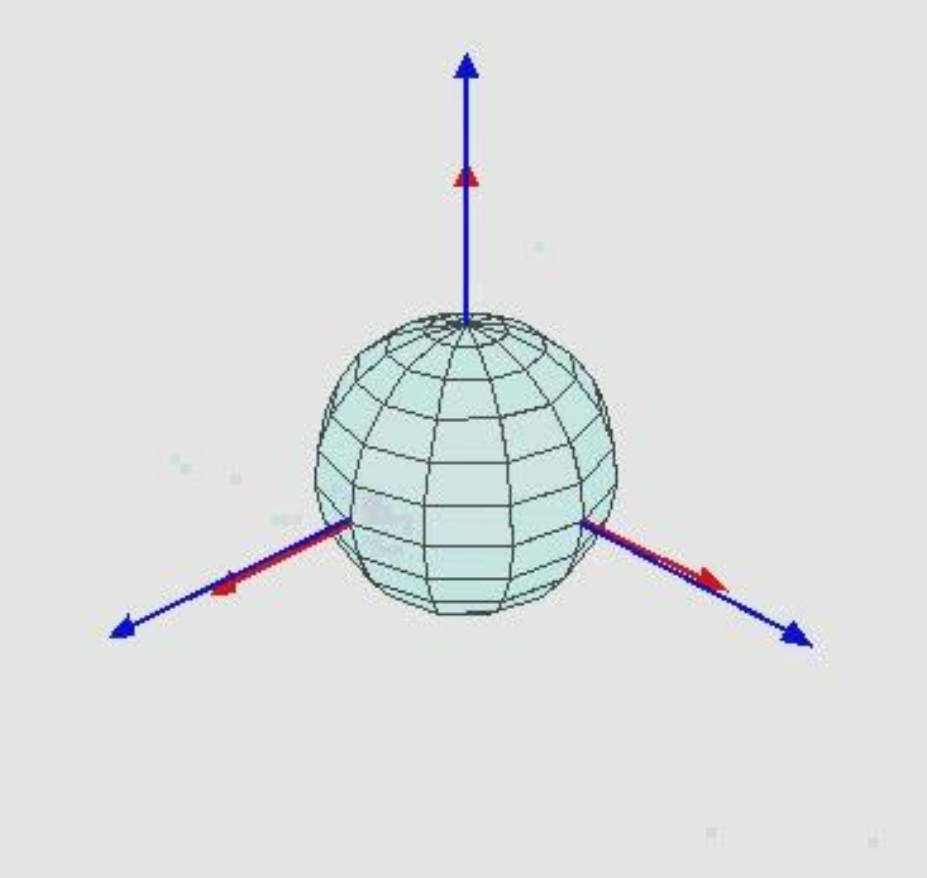
$$x' = x\cos\gamma - y\sin\gamma$$
$$y' = x\sin\gamma + y\cos\gamma$$
$$z' = z$$
$$\mathbf{x}' = \mathbf{R}_z\mathbf{x} = \begin{pmatrix}\cos\gamma & -\sin\gamma & 0\\ \sin\gamma & \cos\gamma & 0\\ 0 & 0 & 1\end{pmatrix}\mathbf{x}$$

- Elementary rotation by β around y:

$$x' = x\cos\beta + z\sin\beta$$
$$y' = y$$
$$z' = -x\sin\beta + z\cos\beta$$
$$\mathbf{x}' = \mathbf{R}_y\mathbf{x} = \begin{pmatrix}\cos\beta & 0 & \sin\beta\\ 0 & 1 & 0\\ -\sin\beta & 0 & \cos\beta\end{pmatrix}\mathbf{x}$$

- Elementary rotation by α around x:

$$x' = x$$
$$y' = y\cos\alpha - z\sin\alpha$$
$$z' = y\sin\alpha + z\cos\alpha$$
$$\mathbf{x}' = \mathbf{R}_y\mathbf{x} = \begin{pmatrix}1 & 0 & 0\\ 0 & \cos\alpha & -\sin\alpha\\ 0 & \sin\alpha & \cos\alpha\end{pmatrix}\mathbf{x}$$

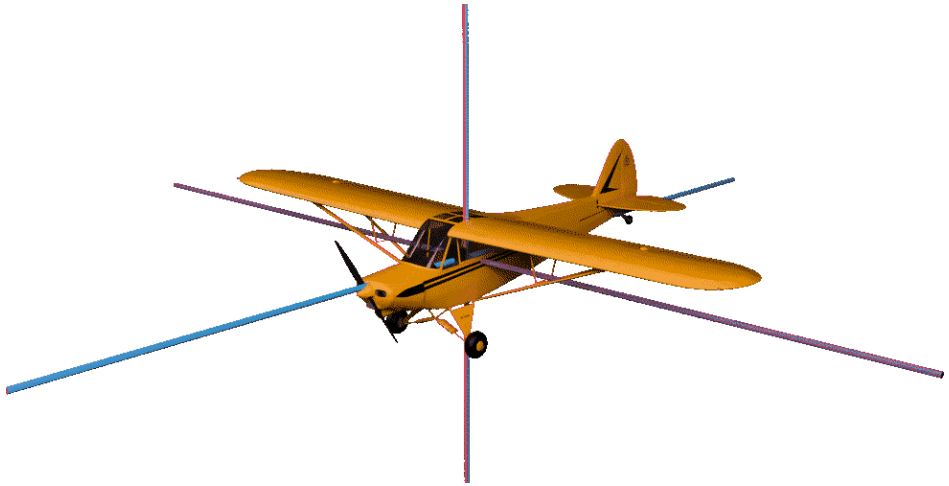


Classical Euler angles -- Source: Wikipedia

- **Any rotation in 3D space can be expressed as the product of 3 successive rotations.**
- **This concept is known as „Euler Angles“ after Leonhard Euler (who didn‘t discover this fact but described it scientifically first in 1775.**
- **There is no single norm on how to do this. Rather, there are several established variants.**
- **Classical Euler angles:**

1. Rotate around z axis by γ_1
2. Rotate around (the new) x‘ axis by α
3. Rotate around (the new) z“ axis by γ_2

$$\mathbf{R} = \mathbf{R_{z''}R_{x'}R_z}$$



Yaw/Pitch/Roll -- Source: Wikipedia

- **Aviation / Automotive – Yaw/Pitch/Roll:**
  1. Rotate around z axis („yaw“) by γ
  2. Rotate around y‘ axis („pitch“) by β
  3. Rotate around x‘‘ axis („roll“) by α

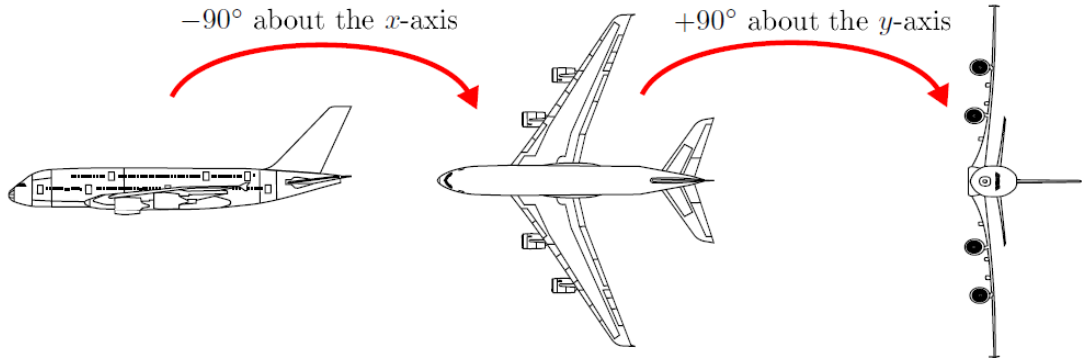
$$\mathbf{R} = \mathbf{R}_{\mathbf{x}''}\mathbf{R}_{\mathbf{y}'}\mathbf{R}_{\mathbf{z}}$$

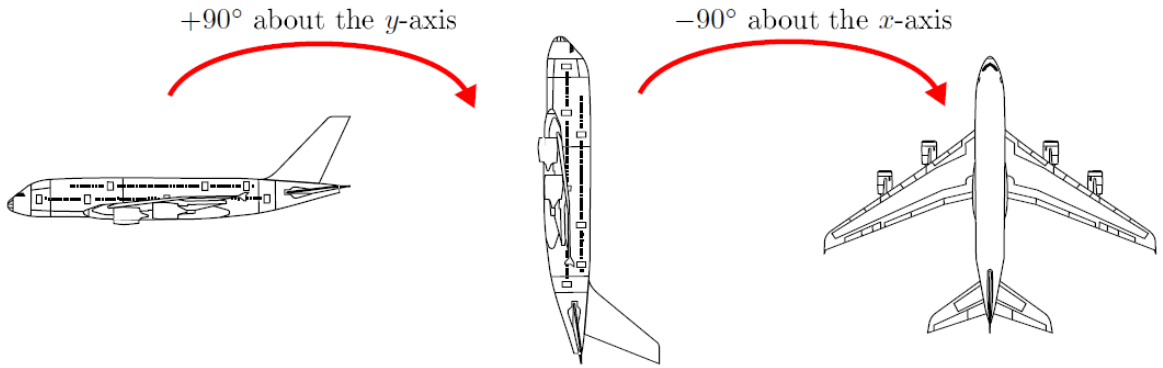
- **Order matters in coordinate transformations!**
  - The order of rotations is important because matrix multiplication is non-commutative.
  - The „first“ axis rotation is rightmost, the „last“ is leftmost!

- Important to note that

$$\boldsymbol{T}_f = \boldsymbol{T}_z \cdot \boldsymbol{T}_y \cdot \boldsymbol{T}_x \neq \boldsymbol{T}_x \cdot \boldsymbol{T}_y \cdot \boldsymbol{T}_z$$

- This can be easily verified graphically
- Apply rotations of $\pm 90^\circ$, start from the same attitude
- Sequence 1:
  - First rotate by -90° about the x-axis
  - Then rotate by $+90^\circ$ about the y-axis
- Sequence 2:
  - First rotate by$+90^\circ$ about the y-axis
  - Then rotate by -90° about the x-axis





Gimbal-Lock Phenomenon of Euler Angles

- Euler angles have a singularity at $\theta = \pm\pi/2$
- Considering the sequence of rotations from the n.e.d. to the body-fixed frame for $\theta = \pi/2$ we have

$$\boldsymbol{T}_n^b = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\Phi & \sin\Phi \\ 0 & -\sin\Phi & \cos\Phi \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \cos\Psi & \sin\Psi & 0 \\ -\sin\Psi & \cos\Psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & -1 \\ \sin\Phi\cos\Psi - \cos\Phi\sin\Psi & \cos\Phi\cos\Psi + \sin\Phi\sin\Psi & 0 \\ \cos\Phi\cos\Psi + \sin\Phi\sin\Psi & \cos\Phi\sin\Psi - \sin\Phi\cos\Psi & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & -1 \\ \sin(\Phi - \Psi) & \cos(\Phi - \Psi) & 0 \\ \cos(\Phi - \Psi) & -\sin(\Phi - \Psi) & 0 \end{bmatrix}$$

- There is an infinite number of combinations of Φ and Ψ representing the same transformation matrix
- No unique mapping from the transformation matrix to the Φ and Ψ

- **Putting translation and rotation together, the full coordinate transform becomes**

$$\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$$

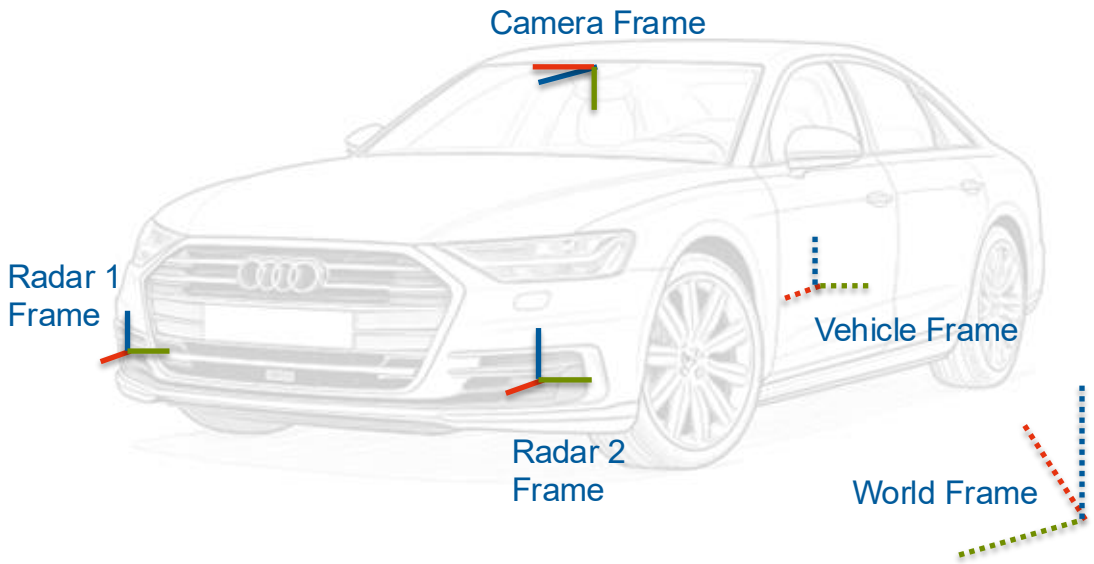
with the Euler rotation matrix R and the translation vector t describing the relationship between coordinate frames.

- **This expression can be simplified by extending coordinates to 4 dimensions by adding a 1:**

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- **Then we can combine R and t like this:**

$$\mathbf{M} = \begin{pmatrix} & \mathbf{R} & & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



- **...and the full equation becomes**

$$\mathbf{x}' = \mathbf{M}\mathbf{x}$$

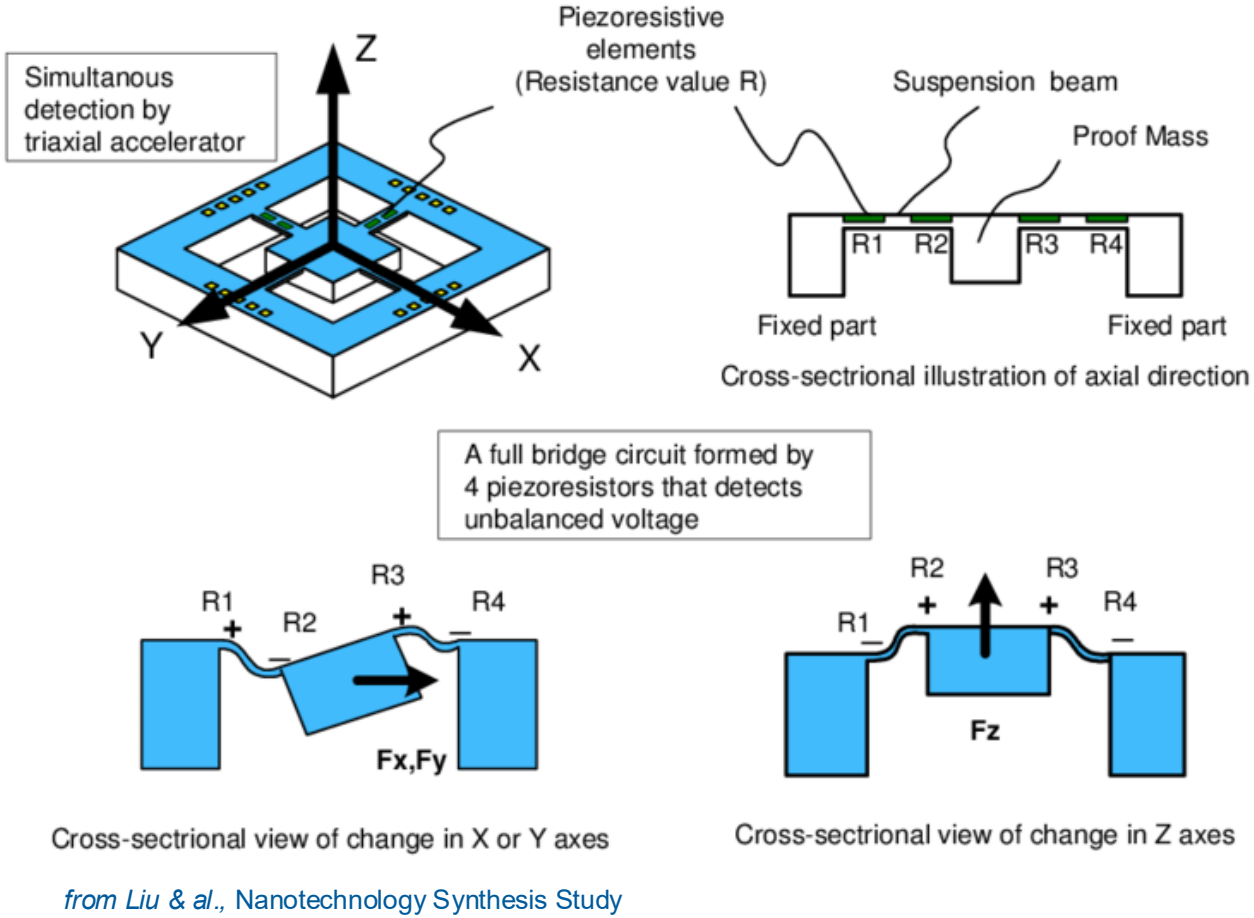
- **We can now easily chain multiple transforms (e.g. world to vehicle with M_v to camera with M_c):**

$$\mathbf{x}' = \mathbf{M}_v\mathbf{M}_c\mathbf{x}$$

Simple Example: Pitch and Roll from Gyro and Accelerometer

(with Complementary Filters, for now)

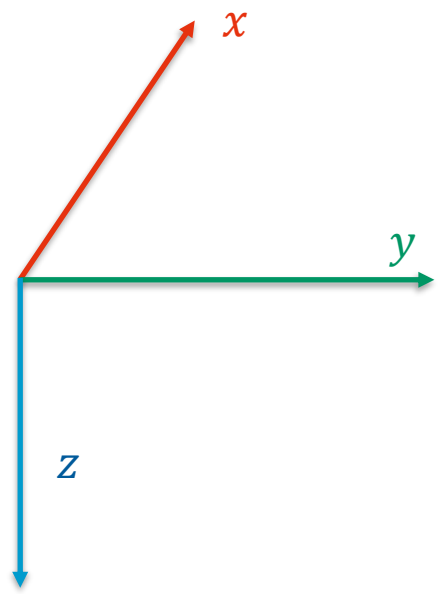
- Task: **Estimate Roll and Pitch for a Robot** (or vehicle, or drone, or or or...)
- Inputs:
  - Accelerometer – Measures **translational acceleration** (e.g. gravity, but also all others)
  - Gyroscope – Measures rotational **velocity** (but no absolute rotation)
- Desired output:
  - 2 degrees of freedom: pitch and roll
  - Free of drift, reasonably free of noise, absolute
- Limit: Can not be absolute in heading without a magnetometer (see later)



from Liu & al., Nanotechnology Synthesis Study

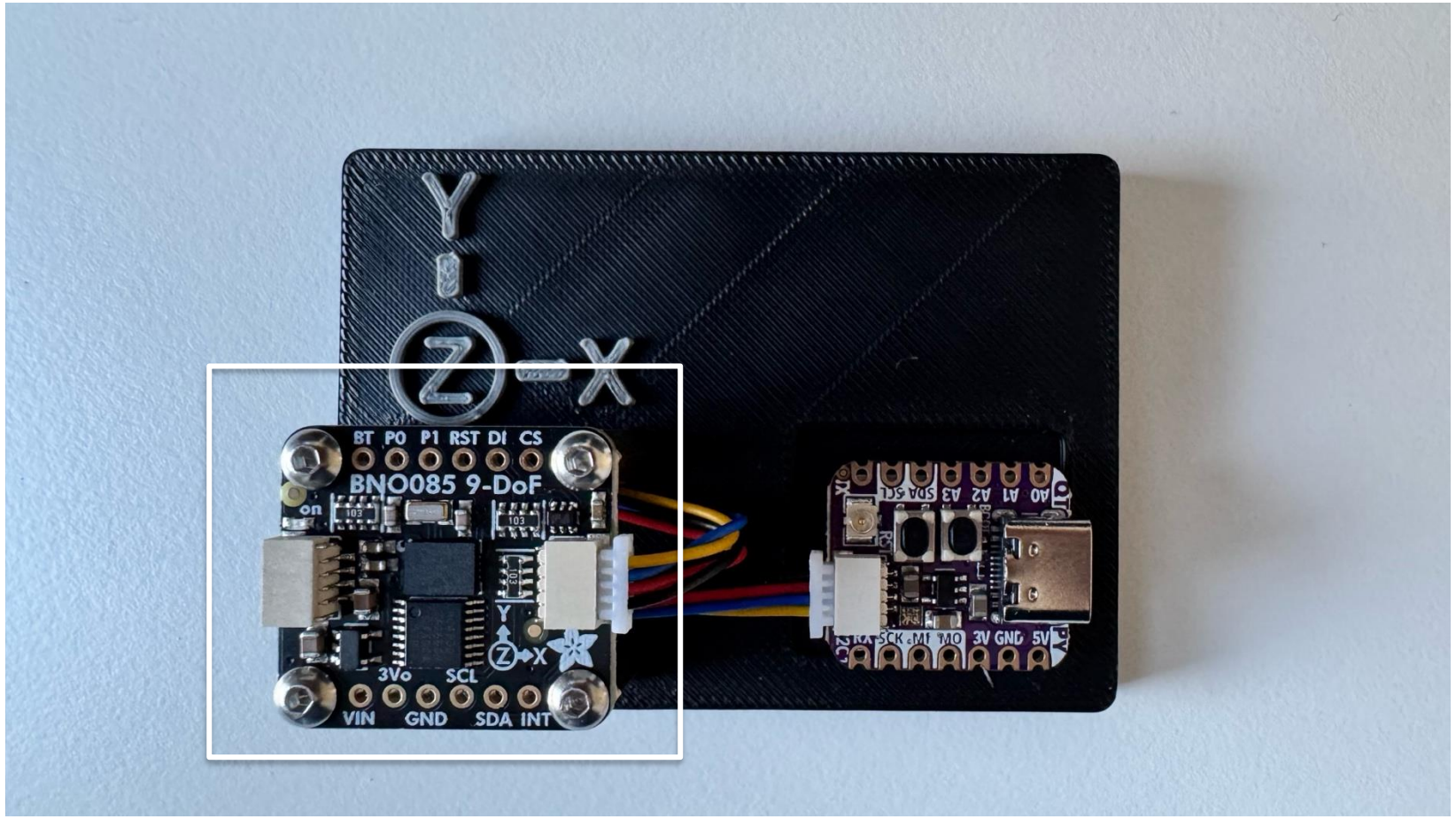
Accelerometer Output

- **Glossary:**
  - Position of a body in space $\mathbf{x} = (x, y, z)$
  - Velocity of a body through space $\mathbf{v} = (v_x, v_y, v_z) = \frac{\partial \mathbf{x}}{\partial t}$
  - Acceleration of a body through space $\mathbf{a} = (a_x, a_y, a_z) = \frac{\partial \mathbf{v}}{\partial t} = \frac{\partial^2 \mathbf{x}}{\partial t^2}$
  - Gravity vector $\mathbf{g} = (0,0, g)$
  - Estimate value $\hat{a}$
  - Derivative over time $\dot{\alpha}$
- **In this example, we are using a coordinate system where z is pointing down to reflect the direction of gravity. As we know, adding a 180° rotation around x at the end will right us up again, so we save it here.**



Sensor Networks and Sensor Fusion

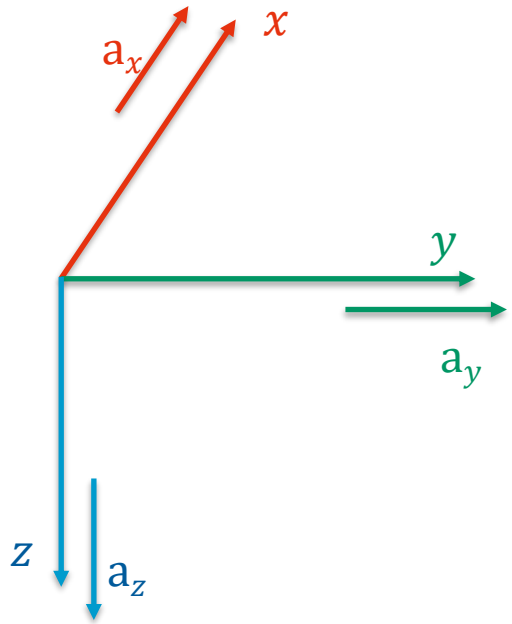
Simple Example: Inertial Measurement Unit



- **Let‘s start with the accelerometer. How can we describe what it measures?**

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \frac{d\mathbf{v}}{dt} + \mathbf{w}_b \times \mathbf{v} - \mathbf{R}\begin{pmatrix} 0 \\ 0 \\ g \end{pmatrix} + \mathbf{b}(t) + \mathbf{n}(t)$$

Measured Acceleration | Linear motion | Rotational motion | Gravity | Time-variant bias | Time-variant noise



- **How do we get roll/pitch estimates, i.e. the matrix R from this?**
- **Let‘s simplify. If the accelerometer is at rest, and we ignore bias and noise, we see**

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \begin{pmatrix} g\sin\beta \\ -g\sin\alpha\cos\beta \\ g\cos\alpha\cos\beta \end{pmatrix}$$

(Why? → exercise! ☺)

- **With** $\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \begin{pmatrix} g \sin\beta \\ -g \sin\alpha \cos\beta \\ g \cos\alpha \cos\beta \end{pmatrix}$ **, we can simplify to**

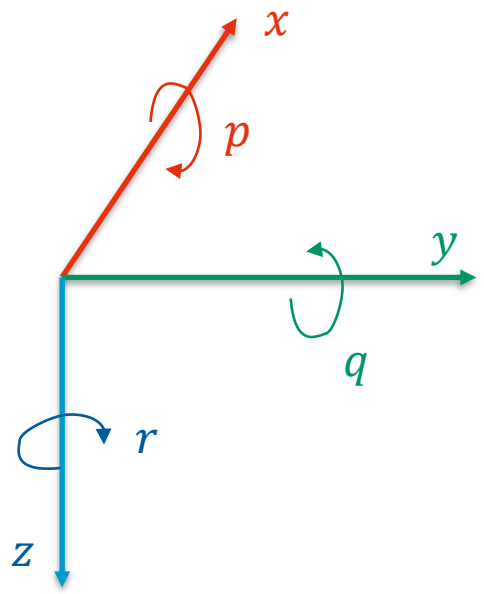
$$\frac{a_y}{a_z} = -\frac{\sin\alpha}{\cos\alpha} = -\tan\alpha \qquad \hat{\alpha}_{\text{acc}} = \tan^{-1}\frac{a_y}{a_z}$$

$$\frac{a_x}{g} = \sin\beta \qquad \hat{\beta}_{acc} = \sin^{-1}\frac{a_x}{g}$$

- **However, this ignores all the other acceleration terms:**
  - actual linear and angular acceleration (kind of important for a vehicle...)
  - time-variant bias due to changes in temperature, humidity etc. – can we calibrate that out?
  - noise – this is typically high frequency and can be reduced by high pass filtering.
- **Let‘s look at how well this works in practice!**

- **Let‘s look at the gyroscope now. It outputs rates of rotation around the body axes.**

$$\begin{pmatrix} \hat{p} \\ \hat{q} \\ \hat{r} \end{pmatrix} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} + \mathbf{b}(t) + \mathbf{n}(t)$$



- **We need to transform to Euler rates to get this into the inertial (world) frame:**

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \end{pmatrix} = \begin{pmatrix} 1 & \sin\alpha\tan\beta & \cos\alpha\tan\beta \\ 0 & \cos\alpha & -\sin\alpha \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix}$$

(Why? → exercise! ☺)

...for which we already need to know α and β...

- **Let‘s simplify and look at the case of initial $\alpha = 0, \beta = 0$.**
  The equation becomes

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \end{pmatrix} = \begin{pmatrix} 1 & \sin 0 \tan 0 & \cos 0 \tan 0 \\ 0 & \cos 0 & -\sin 0 \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} p \\ q \end{pmatrix}$$

- **We could simply integrate from here:**

$$\alpha = \int_0^T \dot{\alpha} dt$$
$$\beta = \int_0^T \dot{\beta} dt$$

or discretely

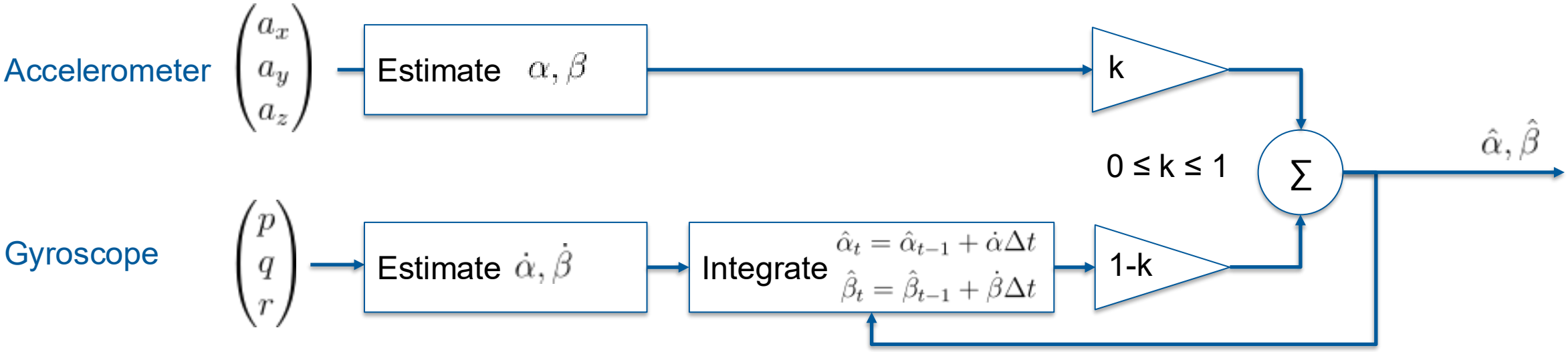
$$\hat{\alpha}_t = \hat{\alpha}_{t-1} + \dot{\alpha}\Delta t$$
$$\hat{\beta}_t = \hat{\beta}_{t-1} + \dot{\beta}\Delta t$$

- **Let‘s see how this works in practice.**

- **What have we learned?**
- **Both accelerometer and gyroscope can give estimates of pitch and roll at certain points.**
  - Accelerometers give good estimates when at rest or over very long periods of time, when accelerations average out and only gravity is left.
  - Gyroscopes give good estimates over very short periods of time, when noise and bias have no time to accumulate.
- **We cannot solve the problem with either of the sensors alone. We need to fuse the data.**

- **Fusing accelerometers and gyroscopes is a well studied problem since the first use of gyroscopes in WW2 rockets.**
- **Many methods have been developed, in order of ascending complexity:**
  - Complementary filters
  - Madgwick or Mahony filters, combining complementary filters with gradient-descent optimization
  - Kalman filters and Extended Kalman filters
  - Neural networks trained on accel/gyro data for various motion profiles.
- **We will look at most of these methods during the course of the lecture, although not always with an IMU in mind.**
- **Today we will look at the simplest variant, the Complementary Filter.**

- **Complementary filters are one of the simplest and most computationally effective fusion methods.**
- **The idea is to use the output of one sensor to correct errors in the output of the other, and vice versa.**
- **Basic principle for our sensor: Weighted sum of individual estimates**



- **Complementary Filter equation:**

$$\hat{\alpha}_n = k\hat{\alpha}_{\text{acc},n} + (1-k)(\hat{\alpha}_{n-1} + \dot{\alpha}_{\text{gyro},n}\Delta t)$$
$$\hat{\beta}n = k\hat{\beta}_{\text{acc},n} + (1-k)(\hat{\beta}_{n-1} + \dot{\beta}_{\text{gyro},n}\Delta t)$$

- **k is a constant which roughly says how much we trust our accelerometer. It is typically close to 0 but we can and should experiment with it.**
- **Let‘s see how this works!**

- **An alternative way of thinking about the Complementary Filter:**
- **We can think of the development of the state of a system that follows a linear, time-invariant model like this:**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1}$$

where

- x is the state vector,
- A is the system model, i.e. “the way things change in the system over time if there is no input“,
- u is a vector of inputs to the system,
- and B is the way how these inputs are influencing our system state.

- **If there were no noise and we could just integrate the gyro, our model can be written as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t & 0\\ 0 & \Delta t\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}}$$

- **However, we know that that is not correct, so we introduce a correction term...**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1} + \mathbf{L}(\mathbf{x}_{n-1} - \mathbf{x}_{\mathrm{meas},\mathbf{n-1}})$$

...again with a model L that governs how the correction term applies to the system state.

- **We can write this now as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t(1-k) & 0\\ 0 & \Delta t(1-k)\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}} + \underbrace{\begin{pmatrix}k & 0\\ 0 & k\end{pmatrix}}_{\mathbf{L}} \begin{pmatrix}\hat{\alpha}_n - \hat{\alpha}_{\mathrm{acc},n-1}\\ \hat{\beta}_n - \hat{\beta}_{\mathrm{acc},n-1}\end{pmatrix}$$

- **The above form is known as a *Luenberger Observer* and is a classic in control theory.**
- **We will come back to this form of thinking about systems (state transition model, input model, correction term) several times, next up with the Kalman filter.**

Thank you very much!